Evaluación de Modelos con Datos Desbalanceados — Detección de Fallas Industriales

Examen del Módulo 3: Modelos de clasificación · Curso de Aprendizaje Supervisado

Objetivo

Aplicar un flujo completo de evaluación de un modelo de clasificación sobre un conjunto de datos fuertemente desbalanceado. Al finalizar el examen, el estudiante deberá ser capaz de: reportar el desempeño de un clasificador con métricas apropiadas para el desbalance (MCC, kappa de Cohen) en lugar de la exactitud; aplicar y comparar distintas estrategias para tratar el desbalance de clases (sobremuestreo, submuestreo, SMOTE y ponderación de clases); y calibrar las probabilidades del modelo final.

Descripción del dataset

El dataset es sintético y simula lecturas de sensores de componentes industriales, con el objetivo de predecir si un componente fallará en las próximas 24 horas. La clase positiva (falla) es poco frecuente (~3% de las observaciones), por lo que el problema debe tratarse como clasificación binaria desbalanceada.

Columna	Descripción
sensor_1 – sensor_12	Doce lecturas numéricas de sensores de un componente industrial (vibración, temperatura, presión, corriente, etc., ya normalizadas por el simulador). No todas son igual de informativas: algunas están correlacionadas entre sí y otras son prácticamente ruido.
falla	Variable objetivo binaria: 1 si el componente falló en las siguientes 24 horas, 0 si no. Es la clase minoritaria (~3% de las filas).

El dataset no se entrega como archivo: se genera dentro del propio notebook con el siguiente código, que debe copiarse exactamente (incluyendo random_state) para que los resultados de todos los estudiantes sean comparables entre sí:

```
from sklearn.datasets import make_classification
import pandas as pd

X, y = make_classification(
    n_samples=6000, n_features=12, n_informative=6, n_redundant=2,
    n_repeated=0, n_clusters_per_class=2, weights=[0.97, 0.03],
    flip_y=0.01, class_sep=0.9, random_state=42)

columnas = [f"sensor_{i}" for i in range(1, 13)]
df = pd.DataFrame(X, columns=columnas)
df["falla"] = y
```

La partición entrenamiento/prueba (80%/20%, estratificada) debe hacerse con train_test_split(..., stratify=y, random_state=42), usando ese mismo random_state=42.

Actividades a desarrollar

1. Preparación de los datos. Generar el dataset con el código anterior y hacer la partición estratificada 80/20. Verificar y reportar el grado de desbalance (conteo y/o gráfico de barras de la variable falla).

2. Modelo base. Entrenar una Regresión Logística (LogisticRegression) sobre los datos escalados (StandardScaler), sin ningún tratamiento de desbalance. Evaluar en el conjunto de prueba reportando exactitud, MCC (matthews_corrcoef) y kappa de Cohen (cohen_kappa_score). En una celda de texto, explicar por qué la exactitud es poco informativa en este problema.
3. Cuatro estrategias contra el desbalance. Para cada una de las siguientes estrategias, entrenar la misma Regresión Logística (mismos hiperparámetros, salvo lo indicado) y reportar (a) el MCC medio ± desviación estándar con validación cruzada estratificada de 5 capas sobre el conjunto de entrenamiento, y (b) MCC y kappa en el conjunto de prueba:
    1. Sobremuestreo aleatorio de la clase minoritaria (oversampling, p. ej. RandomOverSampler).
    2. Submuestreo aleatorio de la clase mayoritaria (undersampling, p. ej. RandomUnderSampler).
    3. SMOTE (Synthetic Minority Oversampling Technique).
    4. class_weight="balanced" en la Regresión Logística, sin remuestrear los datos.
4. Comparación de resultados. Reunir en una tabla las 5 configuraciones (modelo base + las 4 estrategias) con su exactitud, MCC y kappa en el conjunto de prueba. Elegir, con una justificación escrita, la estrategia que recomendarían para este problema.
5. Calibración de probabilidades. Tomar la mejor estrategia de la actividad anterior y comparar sus probabilidades predichas antes y después de aplicar CalibratedClassifierCV (method="sigmoid" o "isotonic"): graficar el diagrama de confiabilidad (calibration_curve) antes y después, y reportar el Brier score (brier_score_loss) antes y después.

Metodología de evaluación

Cada una de las cinco configuraciones (modelo base y las cuatro estrategias) debe evaluarse en dos etapas, claramente diferenciadas en el notebook:

1. Validación cruzada estratificada de 5 capas sobre el conjunto de entrenamiento (4,800 filas), usando MCC como métrica. Reportar la media y la desviación estándar de las 5 capas.
2. Evaluación final en el conjunto de prueba (1,200 filas), entrenando la configuración sobre todo el conjunto de entrenamiento y evaluando una sola vez sobre la prueba. Reportar exactitud, MCC y kappa.

Criterios de evaluación

Rubro	Puntos
Preparación de los datos y modelo base: carga correcta del dataset, partición estratificada, y reporte e interpretación correctos de exactitud, MCC y kappa	20
Sobremuestreo (oversampling): implementación correcta y evaluación con validación cruzada y en prueba	15
Submuestreo (undersampling): implementación correcta y evaluación con validación cruzada y en prueba	15
SMOTE: implementación correcta y evaluación con validación cruzada y en prueba	15
class_weight="balanced": implementación correcta y evaluación con validación cruzada y en prueba	10
Tabla comparativa de las 5 configuraciones y recomendación final fundamentada	10

en la evidencia	
Calibración de probabilidades: diagrama de confiabilidad y Brier score antes/después, correctamente interpretados	15
Total	**100**

Formato y fecha de entrega

- Entregar el notebook con el nombre Apellido_Nombre_Examen_Modulo3.ipynb.
- El notebook debe poder ejecutarse de inicio a fin sin errores (Kernel → Restart & Run All) antes de entregarlo.
- Indicar al inicio del notebook las librerías utilizadas y sus versiones (incluyendo imbalanced-learn).
- Trabajo individual.